6 동작 계획

Motion Planning

이 단원을 마치면

장애물을 피해 스스로 길을 찾고, 블록을 밀지 않고 집어 놓고 쌓는다

먼저 배울 것 2 기구학 & 역기구학 · 5 로봇 제어공학

20차시 **6-1 충돌 검사**
로봇은 가는 도중 어디에 부딪힐까? 움직이기 전에 어떻게 미리 알까?

21차시 **6-2 경로 계획**
벽 너머에 목표가 있다. 로봇은 부딪히지 않는 길을 스스로 찾을 수 있을까?

22차시 **6-3 집기와 놓기**
블록을 밀지 않고 집고 싶다. 원하는 자리에 반듯하게 놓고 싶다. 어떤 순서로 움직여야 할까?

23차시 **6-4 쌓기**
블록 세 개를 탑으로 쌓는다. 얼마나 빨리 움직여도 될까?

이 교재 쓰는 법: 한 주제가 한 차시(50분)

도입 학습 목표를 읽고, **생각 열기** 질문에 내 생각을 적어 본 뒤 그림을 봐요.

개념 정의 · 원리 · 자세히를 읽어요. 처음 보는 기호는 위에 작은 이름이 있고, 주제마다 **기호 표**가 있어요. 대학 과정은 수식까지.

탐구 활동 1 → 2 → 3 시뮬레이터 [교육] 탭에서 같은 동작을 한 가지만 바꿔 세 번 해요. 하기 전에 **예상**을 적고, 해 본 뒤 **기록표**를 채우고**왜 그럴까**를 읽어요. `실행` 처럼 네모 친 말은 화면의 버튼 이름이에요.

정리 · 평가 세 결과를 비교해 **기억할 것**과 **현장에서는**을 확인하고, 문항지로 평가해요.

참고 자료 LaValle, S., Planning Algorithms, Cambridge University Press, 2006, 4장(The Configuration Space)·5장(Sampling-Based Motion Planning) · Lynch, K. & Park, F., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017, 10장(Motion Planning) · Siciliano, B., Sciavicco, L., Villani, L. & Oriolo, G., Robotics: Modelling, Planning and Control, Springer, 2009 · Kuffner, J. & LaValle, S., RRT-Connect: An Efficient Approach to Single-Query Path Planning, IEEE ICRA, 2000 · Kavraki, L., Švestka, P., Latombe, J.-C. & Overmars, M., Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces, IEEE Transactions on Robotics and Automation, 1996 · Şucan, I., Moll, M. & Kavraki, L., The Open Motion Planning Library, IEEE Robotics & Automation Magazine, 2012 · ISO 10218-1/-2 Robotics — Safety requirements for industrial robots (2025 개정판, 이전판 2011), ISO/TS 15066:2016 Collaborative robots (안전 여유·거리) · Elephant Robotics, myCobot 280 사양 (가반하중 250 g, 반복 정밀도 ±0.5 mm, 관절 한계) 및 흡착 펌프·평행 그리퍼 사양

20차시 6-1 충돌 검사

Collision Checking · 단원 안 1 / 4차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

링크는 캡슐로, 물체는 상자로 감싼다. d(거리)와 r(캡슐 반지름)을 견주어 충돌을 판정할 수 있다.

이동 각과 검사 간격으로 검사 횟수를 계산할 수 있다. 간격이 넓으면 얇은 벽을 놓친다(**터널링**). 그 까닭을 말할 수 있다.

안전 여유를 두는 까닭을 설명할 수 있다. 자세 하나를 점 하나로 보는 **형상 공간**을 설명할 수 있다.

Q **생각 열기**

로봇은 가는 도중 어디에 부딪힐까? 움직이기 전에 어떻게 미리 알까?

함께 생각해 보기

눈을 감고 방을 건널 때 출발점과 도착점만 확인하면 안전할까?

로봇은 움직이기 전에 부딪힐 곳을 어떻게 미리 알까?

해 볼 것

☐ 출발과 도착 사이에 얇은 벽이 있는 그림을 그려 본다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

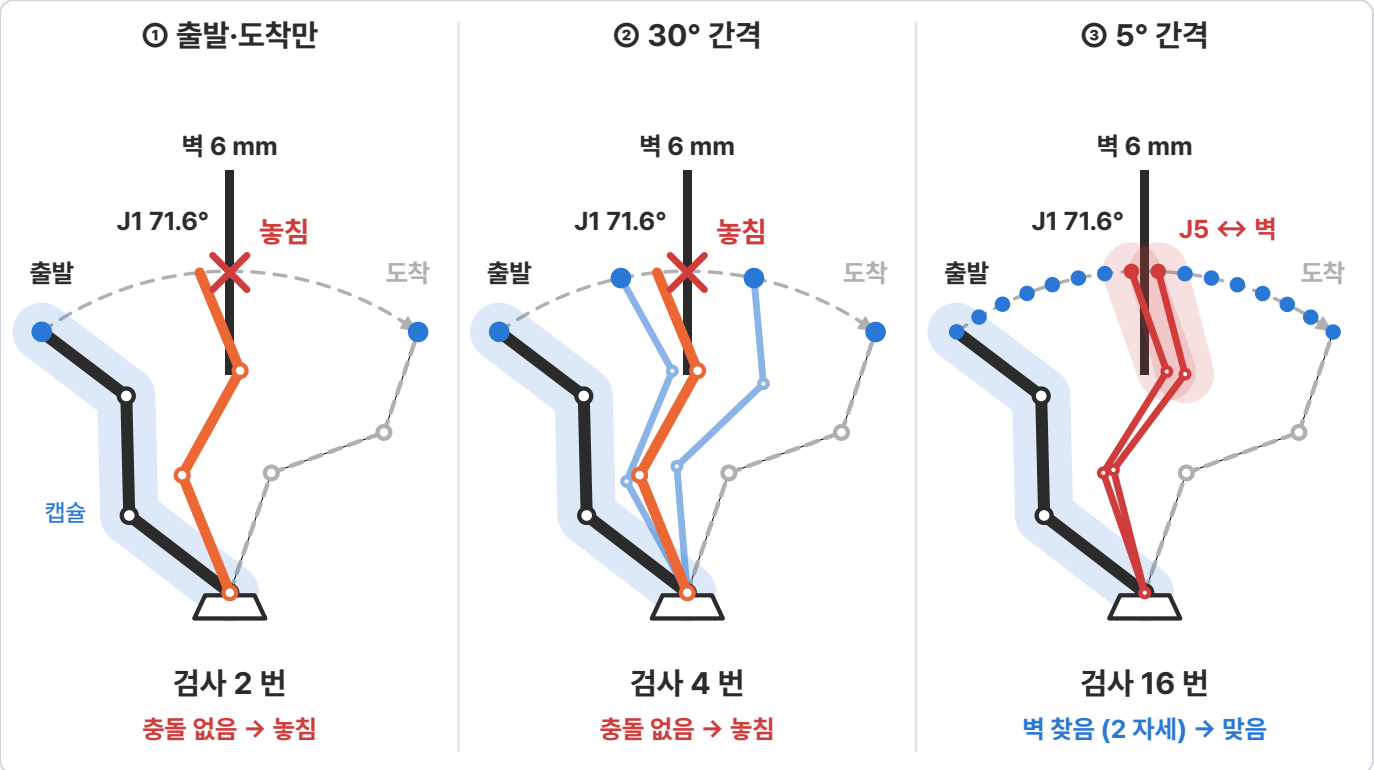


그림 6-1 검사 간격에 따른 충돌 검사: 출발·도착만 · 30° 간격 · 5° 간격 (J1 71.6°, 벽 6 mm)

정의

충돌 검사는 링크와 주변 물체가 겹치는지 계산한다. 둘을 단순한 모양으로 감싸서 계산한다. 움직임은 길 위의 자세를 일정 간격마다 검사한다.

원리

링크는 캡슐(원통 양끝에 반구)로 감싼다. 물체는 상자로 감싼다. 그러면 "거리가 반지름보다 작은가" 하나로 겹침을 안다.

움직임은 길을 일정 간격으로 나눈다. 그 자세들만 검사한다. 검사와 검사 사이는 보지 않는다.

간격이 넓으면 얇은 벽을 건너뛴다(**터널링**). 간격을 좁히면 벽을 잡아낸다. 하지만 검사 횟수가 늘어난다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
d	디	**거리**: 캡슐 가운데 선과 상자 사이의 가장 가까운 거리	mm
r	알	**캡슐 반지름**: 링크를 감싼 캡슐의 굵기(반지름)	mm

개념 자세히

로봇의 3D 모델은 작은 삼각형 수만 개로 되어 있어서, 그대로 서로 겹치는지 비교하면 너무 느리다. 그래서 링크와 물체를 단순한 모양(**경계 부피**)으로 감싼다. 이 시뮬레이터는 링크마다 캡슐을 씌우는데, 반지름은 서보 몸통 폭의 절반쯤인 16~22 mm 이고, 그리퍼도 작은 캡슐로 감싼다. 장애물은 상자로 감싼다. 캡슐의 가운데 선과 상자 사이 거리 $\underset{\text{거리}}{d}$ 가 캡슐 반지름 $\underset{\text{캡슐 반지름}}{r}$ 보다 작으면 둘이 겹친 것, 곧 충돌이다. 책상과 로봇 받침도 같은 방법으로 검사한다.

멈춘 자세 하나를 검사하는 것은 쉽다. 그런데 움직임은 출발에서 도착까지 셀 수 없이 많은 자세를 지나가므로 모두 검사할 수는 없다. 그래서 길을 일정 간격으로 나눠 그 자세들만 검사한다(이산 검사). 실습에서는 J1 을 71.6° 돌려 두께 6 mm 벽 너머로 간다. 출발·도착 두 자세만 검사하면 둘 다 비어 있어 "충돌 없음" 이라고 하지만, 실제로 움직이면 손목(J5)이 벽에 닿는다. 30° 간격(검사 4 번)도 벽을 건너뛰어 놓치고, 5° 간격(검사 16 번)이 되어서야 벽과 겹친 자세 2 개를 찾아낸다. 간격을 좁힐수록 안전하지만 검사 횟수가 늘어 계산이 많아진다. 쓸고 지나간 부피 전체를 한 번에 검사하는 **연속 충돌 검사**도 있다.

계산한 모양은 실제와 조금 다르다. 모델 오차가 있고, 같은 명령에도 도착점이 ±0.5 mm 쯤 달라지며(반복 정밀도), 물체 위치도 정확히 모를 수 있다. 그래서 캡슐을 일정 거리만큼 두껍게 잡아($d < r$ + 여유 이면 경고) 닿기 전에 미리 멈추게 하는데, 이 거리를 **안전 여유**라고 한다. 여유가 너무 크면 실제로는 갈 수 있는 곳도 막혀 버리므로 알맞게 정해야 한다. 사람과 함께 일하는 로봇은 이와 따로 사람과의 거리를 재어, 사람이 가까워지면 느려지거나 멈춘다(ISO/TS 15066 의 속도 · 거리 감시).

로봇의 자세는 관절 각 6 개로 정해지므로, 자세 하나는 "관절 각 6 개로 된 공간"의 점 하나로 볼 수 있다. 이 공간을 **형상 공간**(C-space)이라고 한다. **충돌 검사**는 "이 점이 부딪히지 않는 자유 공간에 있는가?" 를 묻는 것이고, 움직임은 점들을 잇는 선이다. 관절 2 개짜리 팔이라면 가로를 J1, 세로를 J2 로 한 지도에 부딪히는 자세를 칠해 장애물이 형상 공간에서 어떤 덩어리가 되는지 그릴 수 있다. 6-2 에서는 이 공간에서 칠해지지 않은 곳만 지나는 길을 찾는다.

대학 · 수식

충돌(경고) 판정: d(캡슐 축, 상자) < r + 여유 (r: 캡슐 반지름)

검사 자세 수 = ⌈이동 각 / 간격⌉ + 1 → 71.6° 이동: 30° 간격 4 개, 5° 간격 16 개

한 간격 동안 링크 이동 ≈ 회전 반경 × 간격(rad), 이 값이 장애물 두께 + 캡슐 지름(2r) 보다 크면 터널링이 생길 수 있다

형상 공간 C = { $(\theta_1, \ldots, \theta_6)$ }, 자유 공간 C_free = { $\theta \in C$: 충돌 없음 }

탐구 활동 시뮬레이터 [교육] 탭 · 6-1 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 멈춘 자세만

처음 조건 · 검사 간격: 멈춘 자세만 (검사 2 번)

목표	출발과 도착 두 자세만 검사해도 움직이는 도중의 충돌을 잡을 수 있는지 알아본다.
예상	출발 자세와 도착 자세가 모두 벽에 닿지 않는다면, 그 사이를 움직이는 동안에도 안전할까? ☐ 1. 도중에 부딪힐 수 있다 ☐ 2. 끝까지 안전하다
할 일	`멈춘 자세만 검사` 를 누른다. 출발·도착 두 자세만 검사하고 움직인다.
볼 것	검사 2 번, "충돌 없음" 이라고 했는데 움직이다 J5 가 벽에 닿아 멈춘다 (놓침)
왜	출발과 도착은 벽의 양쪽에 있어 둘 다 비어 있지만, 팔은 J1 을 71.6° 돌리는 동안 벽을 가로질러 지나간다. 검사는 두 끝 자세만 보았으므로 가운데에서 생기는 충돌을 알 수 없고, "충돌 없음" 이라고 판정한 채 움직이다 J5 가 벽에 닿는다.
핵심	**두 끝만 검사 → 벽을 놓쳐 J5 가 닿는다**

2 활동 2 · 30° 간격

바꾸는 것 · 검사 간격: 멈춘 자세만 → 30° 간격

목표	길 중간의 자세를 몇 개 더 검사하면 벽을 잡을 수 있는지 알아본다.
예상	30° 간격으로 검사하면 몇 번 검사하게 될까? 두께 6 mm 벽을 찾아낼까? ☐ 1. 검사 2 번, 벽 찾음 ☐ 2. 검사 4 번, 벽 찾음 ☐ 3. 검사 4 번, 벽 놓침
할 일	`검사 간격 30°` 를 누른다
볼 것	검사 4 번. 검사점이 벽을 건너뛰어 "충돌 없음" 이 나오지만, 움직이니 J5 가 벽에 닿는다(놓침)
왜	71.6° 를 30° 가 넘지 않게 똑같이 나누면 3 칸(한 칸 약 24°)이라, 출발을 넣어 ⌈71.6 / 30⌉ + 1 = 4 자세만 검사한다. 검사와 검사 사이에 손목은 수십 mm 를 지나가는데 벽은 6 mm 라, 벽과 겹치는 자세들이 검사점 사이에 쏙 들어가 하나도 검사되지 않는다. 이것이 **터널링**이다.
핵심	**30° 간격, 검사 4 번 → 벽을 건너뛴다**

3 활동 3 · 5° 간격

바꾸는 것 · 검사 간격: 30° 간격 → 5° 간격

목표	간격을 좁히면 벽을 찾는지, 그 대신 무엇이 늘어나는지 확인한다.
예상	5° 간격으로 좁히면 검사 횟수는 몇 번이 될까? 이번에는 벽을 찾을까? ☐ 1. 검사 8 번, 벽 놓침 ☐ 2. 검사 16 번, 벽 찾음 ☐ 3. 검사 16 번, 벽 놓침
할 일	`검사 간격 5°` 를 누른다
볼 것	검사 16 번, 벽 근처 2 자세가 빨갛게 표시되고 "J5 ↔ 벽" 을 찾아 팔이 움직이지 않는다 (맞음)
왜	5° 가 넘지 않게 나누면 15 칸(한 칸 약 4.8°)이라 ⌈71.6 / 5⌉ + 1 = 16 자세를 검사한다. 벽과 겹치는 구간이 J1 으로 10° 남짓이라 5° 마다 검사하면 그 안에 검사점이 두 개 들어가고, 그 두 자세에서 "J5 ↔ 벽" 을 찾아 움직이기 전에 멈춘다. 대신 검사 횟수가 4 번에서 16 번으로 4 배 늘어 계산이 많아진다.
핵심	**5° 간격, 검사 16 번 → 벽을 찾아 멈춘다**

✎ 기록표: 화면의 결과를 적어요

활동	검사 간격(출발·도착만·30°·5°)	검사 횟수(2·4·16 번)	찾은 충돌(없음·없음·2 자세)	판정(놓침·놓침·맞음)
활동 1 멈춘 자세만				
활동 2 30° 간격				
활동 3 5° 간격				

정리: 세 활동 비교하기

세 번의 검사점을 한 화면에 겹쳐 놓는다. 간격·검사 횟수·벽을 찾았는지를 표로 본다.

알게 된 것:

정리 질문

검사 간격을 좁히면 무엇을 얻고 무엇을 잃을까?

내 답:

기억할 것

충돌 검사는 세 가지로 한다. 단순한 모양, 촘촘한 간격, 안전 여유다. 검사하지 않은 사이는 알 수 없기 때문이다.

현장에서는

공장은 로봇 프로그램을 만들거나 바꾸면 먼저 검사한다. 시뮬레이션 소프트웨어에서 충돌 검사를 한다. 로봇 · 공구 · 지그 · 설비를 단순한 모양으로 감싼다. 검사가 끝나면 실제 로봇으로 보낸다. 로봇 제어기에도 안전 기능이 있다. 팔이나 공구가 금지 구역에 들어가려 하면 멈춘다. 검사에서 놓친 움직임도 한 번 더 막는다.

교사용 답

예상 정답: 활동 1: 1. 도중에 부딪힐 수 있다 · 활동 2: 3. 검사 4 번, 벽 놓침 · 활동 3: 2. 검사 16 번, 벽 찾음

정리 질문: 얇은 벽도 놓치지 않아 더 안전해지지만, 검사 횟수가 늘어 계산이 많아진다.

개념 확인 질문: 캡슐과 상자가 겹쳤는지는 어떤 두 값을 견주어 판단할까?

다음 차시 · 21차시 6-2 경로 계획: 벽 너머에 목표가 있다. 로봇은 부딪히지 않는 길을 스스로 찾을 수 있을까?

충돌 검사 collision checking — 로봇 모양이 책상·물체·자기 몸과 겹치는지 계산으로 확인하는 것.

터널링 Tunneling — 검사 간격이 넓어 검사와 검사 사이에 있는 얇은 물체를 건너뛰고 "충돌 없음" 으로 판정하는 현상

안전 여유 safety margin — 닿기 직전이 아니라 앞에서 멈추거나 약하게 누르도록 남겨 두는 여유. 거리나 힘(몸이 견디는 한계 − 누르는 힘)으로 잰다.

경계 부피 Bounding volume — 충돌 검사를 빠르게 하려고 로봇 링크나 물체를 감싸는 단순한 모양(캡슐·구·상자)

연속 충돌 검사 Continuous collision detection (CCD) — 자세 몇 개만이 아니라 움직이며 쓸고 지나간 부피 전체를 검사해 터널링을 막는 방법

형상 공간 Configuration space (C-space) — 관절 각들로 이루어진 공간. 로봇의 자세 하나가 이 공간의 점 하나다 (myCobot 280 은 6 차원)

21차시 6-2 경로 계획

Path Planning (RRT) · 단원 안 2 / 4차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

관절 이동(관절 보간)은 장애물을 피하지 못한다. 그 까닭을 설명할 수 있다. 사람이 찍은 경유점의 장단점도 말할 수 있다.

RRT 는 무작위 자세 q_rand (무작위 자세) 쪽으로 나무를 키운다. 한 걸음 ε (한 걸음) 씩 키워 길을 찾는다. 이 과정을 순서대로 말할 수 있다.

RRT 의 길은 매번 다르고 가장 짧지도 않다. 그 까닭을 설명할 수 있다. **경로 다듬기**가 하는 일도 설명할 수 있다.

Q **생각 열기**

벽 너머에 목표가 있다. 로봇은 부딪히지 않는 길을 스스로 찾을 수 있을까?

함께 생각해 보기

길 안내 앱은 막힌 길을 어떻게 돌아갈까?

로봇은 벽 너머 목표까지 가는 길을 스스로 찾을 수 있을까?

해 볼 것

☐ 필통을 벽 삼아 손이 넘어가는 길을 종이에 그려 본다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

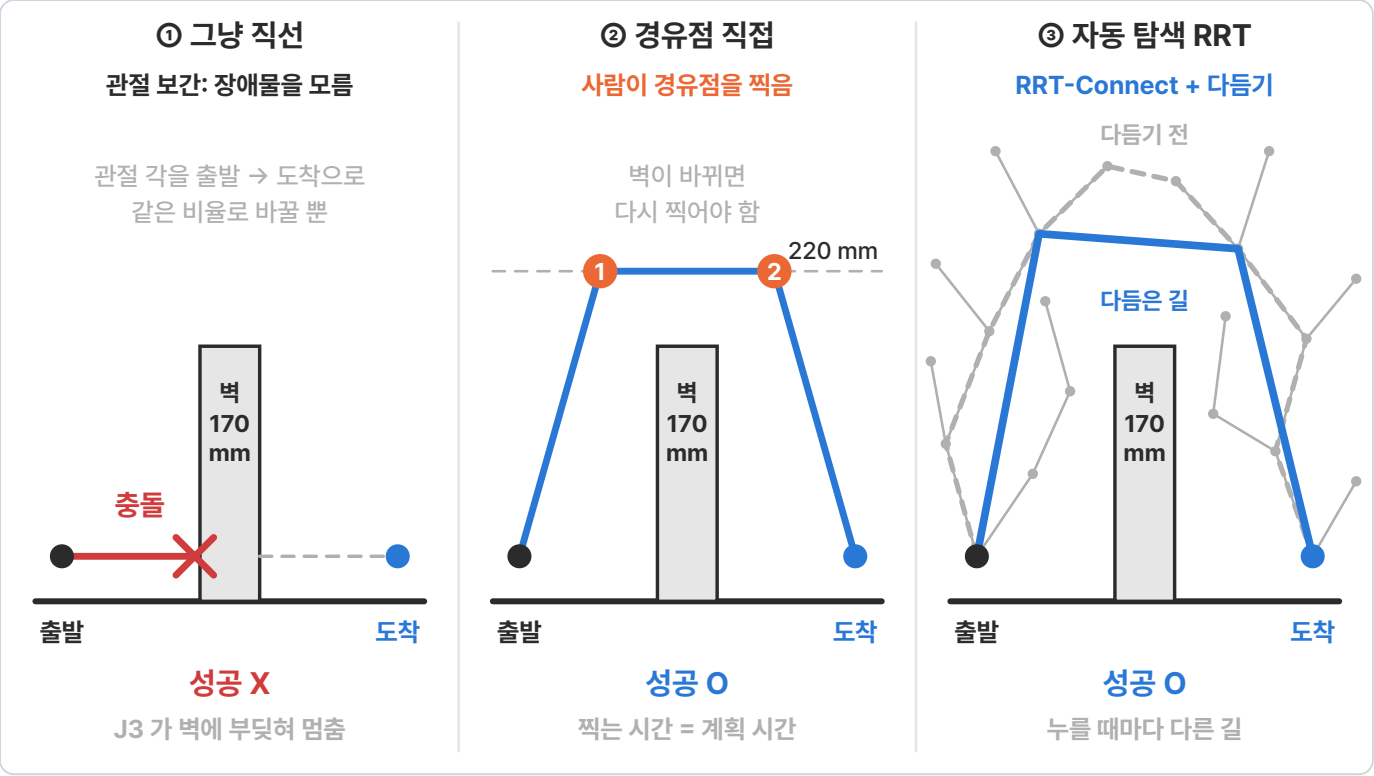


그림 6-2 벽 너머로 가는 세 가지 길: 그냥 직선 · 사람이 찍은 경유점 · RRT 자동 탐색 (벽 170 mm)

정의

경로 계획은 충돌 없는 길을 찾는 것이다. 출발 자세에서 도착 자세까지 간다. RRT 는 그 대표적인 방법이다. 무작위 자세 쪽으로 나뭇가지를 뻗어 길을 찾는다.

원리

관절 이동(관절 보간)은 장애물을 생각하지 않는다. 그래서 부딪힐 수 있다.

사람이 경유점을 찍어 피해 갈 수 있다. 하지만 장애물이 바뀔 때마다 다시 찍어야 한다.

RRT 는 무작위 자세 쪽으로 가지를 조금씩 뻗는다. 충돌 검사를 통과한 가지만 남긴다. 찾은 지그재그 길은 지름길로 다듬는다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
q_rand	큐 랜드	**무작위 자세**: 이번에 무작위로 뽑은 자세(형상 공간의 점)	° (관절 각)
q_near	큐 니어	**가장 가까운 점**: 나무에 이미 있는 점 가운데 무작위 자세에 가장 가까운 점	° (관절 각)
q_new	큐 뉴	**새 점**: 가장 가까운 점에서 무작위 자세 쪽으로 한 걸음 뻗은 점	° (관절 각)
ε	엡실론	**한 걸음**: 가지를 한 번에 뻗는 길이	°

개념 자세히

2 기구학 & 역기구학에서 본 관절 이동(관절 보간)은 모든 관절 각을 같은 비율로 바꿔 가는 가장 단순한 움직임이다. 관절 각은 곧게 바뀌지만 손끝은 곡선으로 간다. 그리고 이 움직임은 장애물을 전혀 모른다. 실습에서는 공구를 아래로 향한 채 높이 170 mm 벽의 한쪽에서 반대쪽 같은 자리로 간다. 화면의 `그냥 직선` 은 이 관절 이동이고, 그대로 가면 J3 가 벽에 부딪혀 멈춘다. 사람이 책상을 클릭하면 그 위 220 mm 에 경유점이 생기고(최대 3 개), 벽 위를 넘어가도록 잘 찍으면 도착한다. 하지만 찍는 데 시간이 들고, 잘못 찍으면 실패하며, 장애물이 바뀌면 처음부터 다시 찍어야 한다.

경로 계획은 6-1 의 **형상 공간**에서 자유 공간만 지나는 길을 찾는 일이다. 관절이 6 개라 6 차원인데, 관절마다 10 칸으로만 나눠도 10^6 = 100만 칸이 되어 칸을 모두 검사하는 방법은 너무 느리다. 그래서 무작위로 자세를 뽑아 충돌 검사만으로 길을 찾는 표본 기반 방법을 쓴다. **RRT** 는 무작위 자세 q_rand(무작위 자세) 를 하나 뽑고, 나무에서 그 자세에 가장 가까운 점 q_near(가장 가까운 점) 를 찾아, 그쪽으로 한 걸음 ε(한 걸음) (이 시뮬레이터는 12°) 뻗은 새 점 q_new(새 점) 를 만든다. q_near 에서 q_new 까지를 2° 마다 충돌 검사해 비어 있으면 나무에 붙이고, 부딪히면 버린다. 실습의 **RRT-Connect** 는 출발과 도착 양쪽에서 나무 두 그루를 키워 서로 만나면 길을 완성한다.

나무로 찾은 길은 무작위로 뻗은 가지를 이었으므로 지그재그다. 그래서 길 위 두 점을 직선으로 이어 보고, 그 직선이 부딪히지 않으면 사이의 점들을 지우는 **경로 다듬기**를 한다. 실습에서 다듬기 전후의 관절 길이를 비교하면 길이 눈에 띄게 줄어든다. `다시 찾기` 는 다른 난수로 다시 뻗으므로 매번 다른 길이 나온다. **RRT** 는 길이 있으면 오래 뻗을수록 찾을 확률이 1 에 가까워지지만(**확률적 완전성**), 가장 짧은 길을 보장하지는 않는다. 계속 다듬어 최단에 가까워지는 RRT* 와, 자유 공간에 점을 미리 뿌려 이어 둔 지도로 여러 번 길을 찾는 **PRM** 도 있다.

경로 계획은 충돌 검사를 아주 많이 쓴다. 실습에서 길을 한 번 찾는 동안 충돌 검사한 자세가 1000 개 안팎이다. 실제 로봇 소프트웨어(ROS 2 의 MoveIt)도 OMPL 이라는 라이브러리로 이런 계획기를 쓴다. 찾은 길(경로)에는 시간이 들어 있지 않으므로, 3 미분기구학 · 궤적 생성에서 본 속도 곡선을 입혀서 움직인다.

대학 · 수식

경로 계획: C_free 안에서 q_start 와 q_goal 을 잇는 연속 곡선을 찾는다

RRT 한 걸음: q_near = 나무에서 q_rand 에 가장 가까운 점, q_new = q_near + ε (q_rand − q_near) / ‖q_rand − q_near‖

다듬기: 길 위 두 점 q_i, q_j 를 잇는 직선이 C_free 안이면 그 사이 점을 지운다

PRM: 자유 공간 표본 N 개 → 가까운 점끼리 충돌 없는 직선으로 이어 그래프(로드맵) → 그래프에서 최단 경로 탐색

칸 나누기 비용: 관절마다 m 칸, 관절 n 개 → m^n 칸 (10 칸, 6 관절 → 10^6)

탐구 활동 시뮬레이터 [교육] 탭 · 6-2 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 관절 이동

처음 조건 · 길 찾는 방법: 관절 이동

목표	장애물을 생각하지 않는 관절 이동(관절 보간)으로 벽 너머 목표에 갈 수 있는지 확인한다.
예상	출발과 도착 사이에 높이 170 mm 벽이 있을 때, 관절 각을 똑바로 바꿔 가면 팔은 벽을 넘어갈까? ☐ 1. 벽을 넘어간다 ☐ 2. 벽에 부딪혀 멈춘다 ☐ 3. 벽을 돌아간다
할 일	`그냥 직선` 을 누른다. 관절 각을 곧게 바꾸는 관절 이동이다.
볼 것	손끝이 벽 쪽으로 곧장 가다 J3 가 벽에 부딪혀 멈춘다(성공 X)
왜	관절 보간은 출발과 도착의 관절 각을 같은 비율로 바꿀 뿐, 그 사이에 팔이 어디를 지나는지 보지 않는다. 출발과 도착이 벽의 양쪽 낮은 곳에 있으므로 그 사이 자세들은 벽을 가로지르게 되고, 팔(J3 쪽)이 벽에 부딪혀 멈춘다.
핵심	**관절 이동 → J3 가 벽에 부딪혀 멈춘다**

2 활동 2 · 경유점 직접

바꾸는 것 · 길 찾는 방법: 관절 이동 → 경유점 직접

목표	사람이 경유점을 찍어 벽을 피하는 방법의 장점과 단점을 알아본다.
예상	경유점을 어디에 몇 개 찍어야 벽을 넘을까? 벽이 다른 자리로 옮겨지면 어떻게 될까? ☐ 1. 벽 양쪽 바로 위에 찍는다 ☐ 2. 출발점 옆에 하나만 찍는다 ☐ 3. 벽에서 멀리 떨어진 곳에 찍는다
할 일	벽 위를 넘어가도록 책상을 클릭해 경유점을 찍는다. 경유점은 최대 3 개, 높이 220 mm 다. 그다음 `경유점으로 가기` 를 누른다.
볼 것	잘 찍으면 벽을 넘어 도착한다. 찍는 데 걸린 시간이 계획 시간으로 기록된다
왜	경유점은 클릭한 곳 위 220 mm 에 생기므로, 170 mm 벽의 양쪽 바로 위에 찍으면 손끝이 벽보다 50 mm 높은 곳을 지나 넘어간다. 사람이 장애물을 보고 길을 정해 준 셈이라 잘 찍으면 성공한다. 하지만 찍는 데 걸린 시간이 그대로 계획 시간이 되고, 잘못 찍으면 경유점 사이에서 부딪히며, 벽이 바뀌면 처음부터 다시 찍어야 한다.
핵심	**경유점 직접 → 벽을 넘지만 사람 손이 든다**

3 활동 3 · 자동 탐색 RRT

바꾸는 것 · 길 찾는 방법: 경유점 직접 → 자동 탐색 RRT

목표	**RRT-Connect** 가 스스로 길을 찾는 모습을 보고, 찾은 길의 성질(매번 다름 · 다듬기)을 확인한다.
예상	`다시 찾기` 를 누를 때마다 같은 길이 나올까? 다듬기 전과 뒤의 길이는 어떻게 다를까? ☐ 1. 매번 같은 길이 나온다 ☐ 2. 매번 다른 길, 다듬으면 길어진다 ☐ 3. 매번 다른 길, 다듬으면 짧아진다
할 일	`RRT 로 길 찾기` 를 누른다. 그다음 `다시 찾기` 를 두세 번 누른다.
볼 것	두 나무가 자라 만나고 다듬은 길로 벽을 넘는다. 누를 때마다 다른 길이 나오고, 다듬기 뒤 관절 길이가 줄어든다
왜	**RRT** 는 무작위 자세 q_rand 를 뽑아 나무에서 가장 가까운 점 q_near 에서 한 걸음 ε (12°) 뻗고, 충돌 검사를 통과한 가지만 남긴다. 출발과 도착 양쪽의 나무가 벽 위에서 만나면 길이 생긴다. 난수가 바뀌면 뽑히는 자세가 달라 나무와 길도 매번 달라지고, 무작위로 뻗은 지그재그 길은 다듬기로 중간 점을 지워 짧아진다.
핵심	**RRT → 스스로 찾지만 매번 다른 길**

✎ 기록표: 화면의 결과를 적어요

활동	성공(X·O·O)	손끝 이동 거리(mm)	계획 시간(없음·찍는 데 걸린 초·ms)	이동 시간(s)
활동 1 관절 이동				
활동 2 경유점 직접				
활동 3 자동 탐색 RRT				

정리: 세 활동 비교하기

세 길을 한 화면에 겹쳐 놓는다. 성공 여부와 손끝 이동 거리를 본다. 계획 시간과 이동 시간도 표로 본다.

알게 된 것:

정리 질문

사람이 찍은 경유점 대신 RRT 를 쓰면 좋은 때는 언제일까?

내 답:

기억할 것

경로 계획은 충돌 검사를 수없이 써서 길을 찾는다. RRT 는 빠르다. 하지만 매번 다르고 가장 짧지는 않다. 그래서 다듬어 쓴다.

현장에서는

로봇 소프트웨어 ROS 2 에는 MoveIt 이 있다. MoveIt 은 OMPL 의 RRT 계열 계획기를 쓴다. 기본값은 RRT-Connect 다. 이것으로 충돌 없는 길을 찾는다. 물류 창고의 상자 집기에 이런 자동 경로 계획을 쓴다. 물체 위치와 주변이 자주 바뀌는 작업이기 때문이다. 자동차 용접 라인의 로봇은 다르다. 같은 동작을 수없이 되풀이한다. 그래서 사람이 경유점을 한 번 가르쳐(티칭) 둔다. 그리고 그대로 반복하는 경우가 많다.

교사용 답

예상 정답: 활동 1: 2. 벽에 부딪혀 멈춘다 · 활동 2: 1. 벽 양쪽 바로 위에 찍는다 · 활동 3: 3. 매번 다른 길, 다듬으면 짧아진다

정리 질문: 물체 위치나 장애물이 자주 바뀔 때다. RRT 는 그때마다 스스로 부딪히지 않는 길을 다시 찾는다.

개념 확인 질문: 경로 계획은 무엇을 찾는 일일까?

다음 차시 · 22차시 6-3 집기와 놓기: 블록을 밀지 않고 집고 싶다. 원하는 자리에 반듯하게 놓고 싶다. 어떤 순서로 움직여야 할까?

경로 계획
path planning
부딪히지 않고 목표까지 가는 길을 찾는 것.

RRT
rapidly-exploring random tree
무작위 자세 쪽으로 나뭇가지를 뻗어 가며 길을 찾는 방법. 충돌하는 가지는 버린다.

경로 다듬기
Path shortcutting
찾은 길 위 두 점을 직선으로 이어 부딪히지 않으면 사이의 점을 지워 길을 짧게 만드는 일

PRM
Probabilistic roadmap
자유 공간에 무작위 점을 미리 뿌려 가까운 점끼리 이어 둔 지도로 길을 찾는 방법

형상 공간
Configuration space (C-space)
관절 각들로 이루어진 공간. 로봇의 자세 하나가 이 공간의 점 하나다 (myCobot 280 은 6 차원)

RRT-Connect
RRT-Connect
출발과 도착 양쪽에서 RRT 나무를 키워 서로 만나게 해 길을 찾는 방법. 한쪽만 키울 때보다 빠르다

확률적 완전성
Probabilistic completeness
길이 있으면 표본을 늘릴수록 찾을 확률이 1 에 가까워지는 성질. 최단 경로는 보장하지 않는다

22차시 6-3 집기와 놓기

Pick and Place, Grasp Planning · 단원 안 3 / 4차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

파지 계획에서 정할 세 가지를 말할 수 있다. 파지점 · 접근 방향 · 도구다. 이 실습은 **그리퍼** 대신 **흡착** 펌프를 쓴다. 그 까닭을 설명할 수 있다.

집기의 네 단계를 순서대로 말할 수 있다. 접근 → 내리기 → 잡기 → 들기 순서다. 순서를 건너뛰면 생기는 일을 설명할 수 있다.

h (놓는 높이) 와 v (부딪는 속도) 의 관계를 말할 수 있다. $v = \sqrt{2gh}$ (g: 중력가속도) 로 놓는 높이를 정할 수 있다.

Q 생각 열기

블록을 밀지 않고 집고 싶다. 원하는 자리에 반듯하게 놓고 싶다. 어떤 순서로 움직여야 할까?

함께 생각해 보기

인형 뽑기 기계는 인형을 어떤 순서로 집을까?

컵을 집을 때 손은 어느 쪽에서 다가갈까?

해 볼 것

☐ 지우개를 집어 옆자리에 옮겨 놓으며 손이 움직인 순서를 적는다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

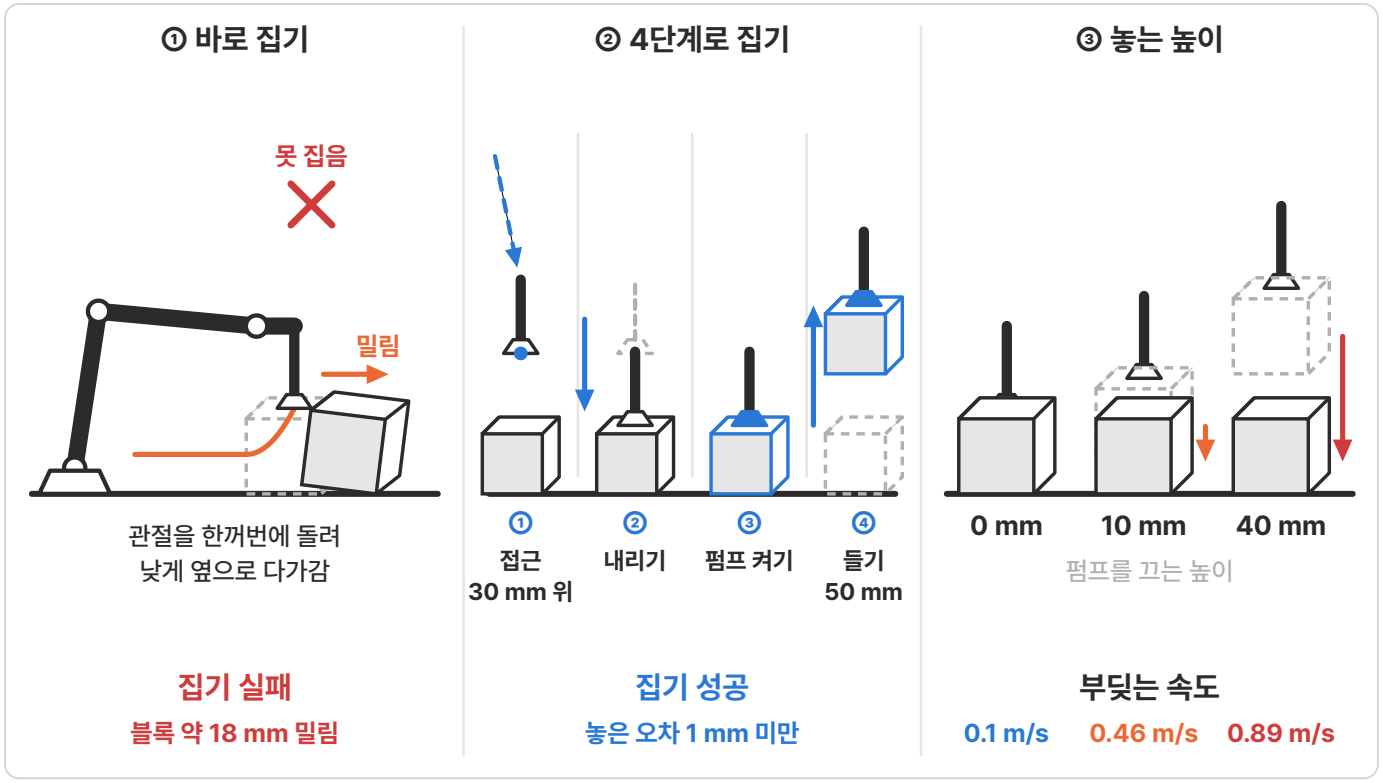


그림 6-3 블록을 집고 놓는 순서와 놓는 높이: 바로 집기 · 4단계로 집기 · 놓는 높이 0 · 10 · 40 mm

정의

집기와 놓기는 엔드 이펙터로 물체를 잡아 옮긴다. 어디를 어느 방향에서 무엇으로 잡을지 정한다. 이것을 파지 계획이라 한다. 기본 순서는 접근 → 내리기 → 잡기 → 들기다.

원리

블록으로 곧장 가면 팔이 낮게 옆에서 다가간다. 그러면 블록을 밀어 버린다.

먼저 블록 위 **접근점**까지 간다. 수직으로 내려가 잡고, 수직으로 든다. 그러면 주변을 건드리지 않는다.

높이 들고 놓으면 떨어지며 빨라져 세게 부딪는다. 그래서 놓을 곳 몇 mm 위에서 낮게 놓는다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
h	에이치	**놓는 높이**: 펌프를 끌 때 블록 바닥과 바닥면 사이 높이	mm (식에서는 m)
v	브이	**부딪는 속도**: 떨어진 블록이 바닥에 닿는 순간의 빠르기	m/s
g	지	**중력가속도**: 떨어지는 물체가 1 초마다 빨라지는 정도, 약 9.8	m/s²
t	티	**낙하 시간**: 펌프를 끈 뒤 바닥에 닿기까지 걸린 시간	s
x	엑스	**앞뒤 좌표**: 로봇 받침에서 책상 앞쪽으로 잰 위치	mm
y	와이	**좌우 좌표**: 로봇 받침에서 옆으로 잰 위치	mm
z	제트	**높이 좌표**: 책상 면에서 위로 잰 높이	mm

개념 자세히

물체를 집기 전에 먼저 정할 것이 있다. 물체의 어디를 잡을지(파지점), 어느 방향에서 다가갈지(접근 방향), 무엇으로 잡을지(도구)다. 이것을 정하는 일을 **파지 계획**이라고 한다. 도구에 따라 답이 다르다. **그리퍼**는 물체의 양 옆면을 쥐므로 물체 폭이 그리퍼가 쥘 수 있는 폭 안에 있어야 한다. 이 키트의 어댑티브 그리퍼(플랜지에서 끝까지 112 mm)는 폭 20~45 mm 인 물체를 쥐므로, 실물에서는 25 mm 블록도 옆으로 쥘 수 있다. 하지만 화면의 연습용 그리퍼는 20 mm 까지만 벌어져서 25 mm 블록을 쥘 수 없다. **흡착** 펌프는 평평한 윗면에 붙으므로 위치가 조금 어긋나도 잘 붙는다. 그래서 이 실습은 흡착 펌프(플랜지에서 흡착컵 끝까지 89 mm, 카메라 플랜지를 끼우면 106 mm)로 25 mm 블록의 윗면을 위에서 집는다. 들 수 있는 무게는 로봇의 가반하중 250 g 안이어야 한다.

집기의 기본은 네 단계다. 첫째, 접근: 블록 윗면 30 mm 위의 **접근점**으로 간다. 둘째, 내리기: 손끝을 수직 직선으로 내린다(2 기구학 & 역기구학의 직선 이동). 셋째, 잡기: **흡착**컵이 윗면에 닿으면 펌프를 켠다. 넷째, 들기: 수직으로 50 mm 들어 올린다. 활동 1의 **바로 집기** 는 이 순서를 건너뛰고 낮은 자세에서 관절을 한꺼번에 돌려 블록 위로 곧장 간다. 그러면 손끝이 낮게 휘어 지나가며 블록을 옆으로 밀어 버리고, 흡착컵이 블록에 닿지 않아 집지 못한다.

놓기는 같은 순서를 거꾸로 한다. 놓을 곳 위로 옮기고, 수직으로 내리고, 펌프를 끄고, 수직으로 빠진다. 이때 놓는 높이가 중요하다. 펌프를 끄면 블록은 그 높이 h (놓는 높이) 에서 자유 낙하하므로, 높을수록 빨라진 채 책상에 부딪는다. 부딪는 속도는 v (부딪는 속도) $= \sqrt{(2\ g\ h)}$ (g (중력가속도) 는 중력가속도 9.8 m/s²)로 높이의 제곱근에 비례한다. 실습에서 0 · 10 · 40 mm 위에서 놓으면 부딪는 속도가 약 0.1 · 0.46 · 0.89 m/s 로 커진다(0 mm 에서도 0.1 m/s 인 것은 **흡착**컵에 매달린 블록이 0.5 mm 쯤 떠 있기 때문이다). 시뮬레이션의 블록은 거의 튕기지 않게 되어 있어 위치 오차가 1 mm 안팎으로 작지만, 실물 블록은 튕기거나 기울 수 있다. 반대로 너무 낮게 내리면 블록을 바닥이나 아래 블록에 밀어 넣는다. 그래서 실제 로봇은 놓을 곳의 높이(쌓은 높이 포함)를 알고 몇 mm 위에서 놓는다.

산업 현장에서 가장 흔한 로봇 일이 바로 집어서 옮기기다. 성공을 정하는 것은 힘이나 속도보다 순서다. 6-4 쌓기에서는 이 순서를 블록 세 개에 되풀이하면서, 층마다 놓는 높이를 25 mm 씩 올린다.

대학 · 수식

접근점 = 파지점 + (0, 0, h_접근) (실습: h_접근 = 30 mm)

낙하 속도 v = √(2 g h) → h = 40 mm 에서 0.89 m/s, 10 mm 에서 0.44 m/s (화면 값은 매달린 높이가 조금 더해져 약간 크다)

낙하 시간 t = √(2 h / g) → h = 40 mm 에서 약 0.09 s

그리퍼로 쥘 수 있는 조건: 쥘 수 있는 가장 작은 폭 ≤ 물체 폭 ≤ 가장 큰 폭 (화면의 연습용 그리퍼 20 mm < 블록 25 mm → 불가, 키트의 어댑티브 그리퍼 20~45 mm → 가능)

탐구 활동 시뮬레이터 [교육] 탭 · 6-3 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 바로 집기

처음 조건 · 집는 방법: 바로 집기

목표	접근 순서 없이 블록을 잡는 자세로 곧장 가면 어떤 일이 생기는지 알아본다.
예상	관절을 한꺼번에 돌려 블록을 잡는 자세로 곧장 가면, 손끝은 블록 위에서 수직으로 내려올까? ☐ 1. 낮게 옆에서 다가가 블록을 민다 ☐ 2. 블록 위에서 수직으로 내려온다
할 일	바로 집기 를 누른다
볼 것	팔이 관절을 한꺼번에 돌려 낮게 다가가다 블록을 약 18 mm 밀어 버리고, **흡착**컵이 닿지 않아 집기에 실패한다
왜	관절을 한꺼번에 돌리면 손끝은 직선이 아니라 휘어진 길로 움직인다. 책상 가까이 있던 팔이 블록 윗면보다 낮게 옆에서 다가가므로, **흡착**컵이 윗면에 닿기 전에 블록 옆면을 쳐서 약 18 mm 밀어 버린다. 블록이 밀려난 자리에 흡착컵이 도착하니 붙을 것이 없어 집기에 실패한다.
핵심	**바로 집기 → 블록을 밀어 집기 실패**

2 활동 2 · 4단계로 집기

바꾸는 것 · 집는 방법: 바로 집기 → 4단계로 집기

목표	접근 → 내리기 → 잡기 → 들기 네 단계로 집으면 결과가 어떻게 달라지는지 확인한다.
예상	블록 위 30 mm **접근점**에 먼저 간 뒤 수직으로 내려가면, 블록이 밀릴까? ☐ 1. 밀린다 ☐ 2. 밀리지 않는다
할 일	4단계로 집기 를 누른다. 먼저 위 30 mm 에 접근해 수직으로 내린다. 펌프를 켜고 수직으로 든다.
볼 것	블록이 제자리에서 붙어 올라오고, 놓을 곳에 1 mm 도 안 되는 오차로 놓인다
왜	**접근점**은 블록 윗면보다 30 mm 높으므로 그곳까지 가는 동안 손끝이 블록에 닿지 않는다. 거기서 손끝을 수직 직선으로 내리면 **흡착**컵이 윗면에 똑바로 닿고, 펌프를 켜 붙인 뒤 수직으로 50 mm 들어 올리므로 블록이 옆으로 밀릴 일이 없다. 그래서 놓을 곳에도 1 mm 가 안 되는 오차로 놓인다.
핵심	**4단계로 집기 → 성공, 오차 1 mm 안쪽**

3 활동 3 · 놓는 높이

바꾸는 것 · 놓는 높이: 2 mm → 0 · 10 · 40 mm

목표	펌프를 끄는 높이에 따라 블록이 책상에 부딪는 속도가 어떻게 바뀌는지 잰다.
예상	놓는 높이를 10 mm 에서 40 mm 로 4 배 올리면, 부딪는 속도도 4 배가 될까? ☐ 1. 그대로다 ☐ 2. 4 배가 된다 ☐ 3. 약 2 배가 된다
할 일	놓는 높이 슬라이더를 0 → 10 → 40 mm 로 바꾼다. 바꿀 때마다 놓기 를 누른다.
볼 것	높을수록 떨어져 부딪는 속도가 커지고(40 mm 에서 약 0.89 m/s), 블록 바닥 높이 그래프가 그 높이에서 0 으로 떨어진다
왜	펌프를 끄면 블록은 높이 h (놓는 높이) 에서 자유 낙하해 $v = \sqrt{2gh}$ (v: 부딪는 속도, g: 중력가속도) 로 부딪는다. 속도는 높이의 제곱근에 비례하므로 높이가 10 → 40 mm 로 4 배가 되면 속도는 0.46 → 0.89 m/s 로 약 2 배가 된다. 0 mm 에서도 0.1 m/s 인 것은 **흡착**컵에 매달린 블록 바닥이 책상보다 0.5 mm 쯤 떠 있기 때문이다.
핵심	**높이 4 배 → 부딪는 속도 약 2 배**

✎ 기록표: 화면의 결과를 적어요

활동	집기(실패·성공·성공)	놓은 위치 오차(mm)	블록 기울기(°)	부딪는 속도(m/s)
활동 1 바로 집기				
활동 2 4단계로 집기				
활동 3 놓는 높이				

정리: 세 활동 비교하기

세 번이 끝난 블록을 책상 위에 나란히 놓는다. 성공 여부·위치 오차·부딪는 속도를 표로 본다.

알게 된 것:

정리 질문

블록을 밀지 않고 집어 반듯하게 놓으려면 어떻게 움직여야 할까?

내 답:

기억할 것

위에서 접근하고, 수직으로 내리고 들고, 낮게 놓는다. 이 순서가 성공을 정한다.

현장에서는

물류 센터에서 상자를 집는 로봇이 있다. 식품 · 전자 부품 포장 라인의 로봇도 있다. 이들은 흡착 패드나 그리퍼로 물건을 집는다. 대부분 같은 순서를 따른다. 위에서 접근해 수직으로 내린다. 잡은 뒤에는 수직으로 든다. 깨지기 쉬운 물건은 놓을 면 바로 위에서 속도를 줄인다. 그리고 천천히 내려놓는다. 흡착을 끈 뒤에는 잠시 기다렸다가 위로 빠진다. 그래야 물건이 끌려 올라오지 않는다.

교사용 답

예상 정답: 활동 1: 1. 낮게 옆에서 다가가 블록을 민다 · 활동 2: 2. 밀리지 않는다 · 활동 3: 3. 약 2 배가 된다

정리 질문: 위에서 접근해 수직으로 내리고, 잡은 뒤 수직으로 들고, 놓을 곳 몇 mm 위에서 낮게 놓는다.

개념 확인 질문: 파지 계획에서 정해야 할 세 가지는 무엇일까?

다음 차시 · 23차시 6-4 쌓기: 블록 세 개를 탑으로 쌓는다. 얼마나 빨리 움직여도 될까?

엔드 이펙터 end effector
팔 끝에 다는 도구. 그리퍼, 흡착 펌프, 펜 등이 있다.

흡착 suction
공기를 빨아들여 물체를 붙잡는 방식. 흡착 펌프와 흡착컵.

접근점 Approach point
물체 바로 위 일정 높이에 정해 두는 점. 여기서 수직으로 내려가 잡는다

그리퍼 Gripper
손가락처럼 물체의 양 옆면을 쥐어 잡는 엔드 이펙터. 물체 폭이 벌어지는 폭 안이어야 잡는다

파지 계획 Grasp planning
물체의 어디를(파지점), 어느 방향에서(접근 방향), 무엇으로(도구) 잡을지 정하는 일

공구 중심점 Tool centre point (TCP)
작업 기준이 되는 공구 끝의 점. 화면의 그리퍼는 플랜지에서 58 mm, 실물 키트의 어댑티브 그리퍼는 112 mm

23차시 6-4 쌓기

Stacking · 단원 안 4 / 4차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

블록 하나를 쌓을 때 거치는 계산 순서를 말할 수 있다. 먼저 위치 → **역기구학** → 충돌 없는 길을 구한다. 그다음 시간 곡선을 입히고 **토크**를 확인한다.

같은 길을 절반 시간에 가면 가속도가 4 배가 된다. 가속도는 $\frac{D}{T^2}$ (이동 거리 / 이동 시간^2)에 비례하기 때문이다. 이것으로 빠르기와 **토크**의 관계를 설명할 수 있다.

걸린 시간 · 최대 **토크** · 탑 기울기를 비교할 수 있다. 그리고 알맞은 속도를 고르고 그 까닭을 말할 수 있다.

Q **생각 열기**

블록 세 개를 탑으로 쌓는다. 얼마나 빨리 움직여도 될까?

함께 생각해 보기

컵 쌓기를 서두르면 왜 탑이 자주 무너질까?

로봇은 얼마나 빨리 움직여도 될까?

해 볼 것

☐ 지우개 세 개를 한 번은 천천히, 한 번은 빨리 쌓아 본다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

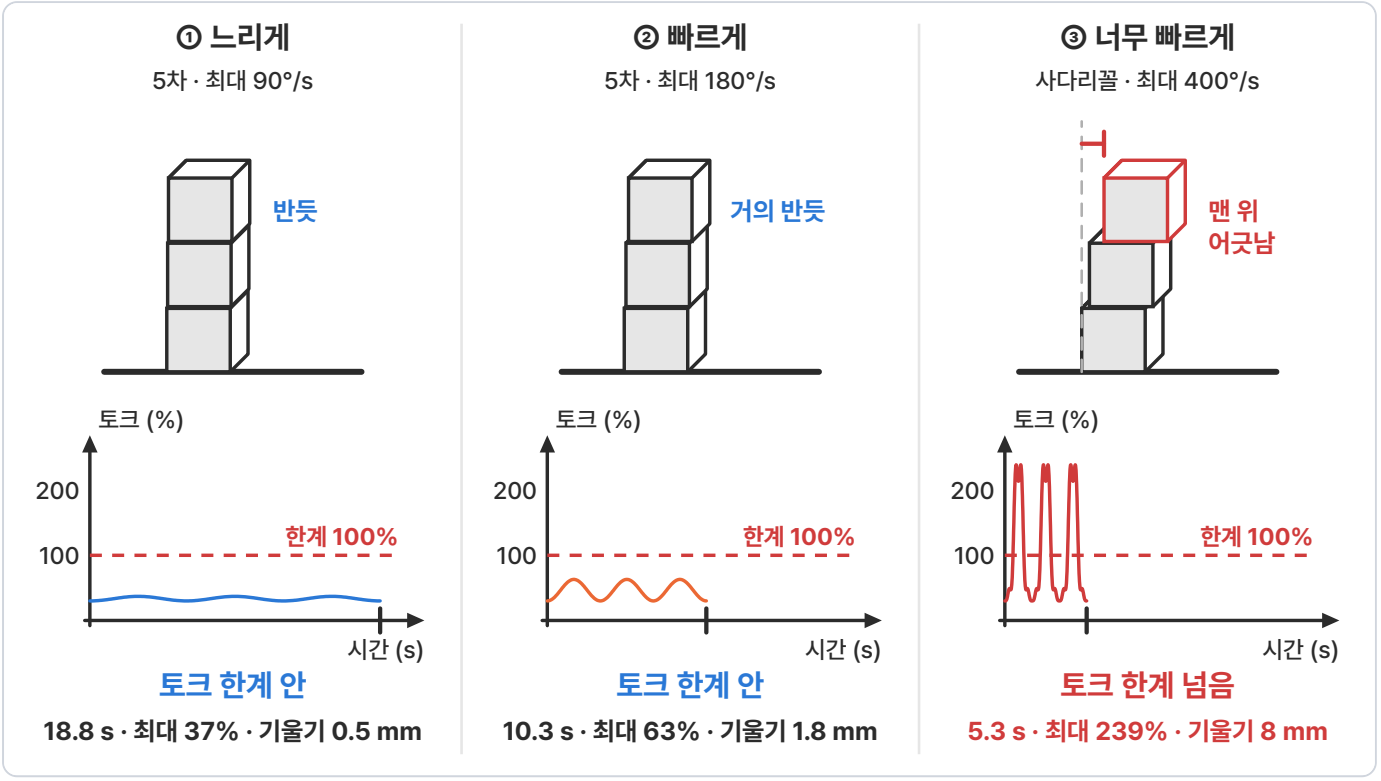


그림 6-4 빠르기에 따른 블록 세 개 쌓기: 느리게 · 빠르게 · 너무 빠르게의 탑, 걸린 시간과 토크(한계 100%)

정의

쌓기는 블록마다 집기·옮기기·놓기를 되풀이한다. 층마다 놓는 높이를 올린다. 빠를수록 사이클 시간은 준다. 하지만 토크와 흔들림이 커져 탑이 어긋난다.

원리

블록마다 위치를 정하고 **역기구학**으로 자세를 구한다. 충돌 없는 길을 찾고 시간 곡선을 입힌다. **토크**를 확인한 뒤 물리 결과를 본다.

빨리 움직이면 가속도가 커져 **토크**가 커진다. 급히 설 때는 팔이 목표를 지나쳤다 돌아오며 흔들린다.

놓는 순간의 흔들림이 블록을 밀어 탑이 어긋난다. 어긋남은 위로 갈수록 쌓인다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
D	디	**이동 거리**: 관절이 한 번에 움직이는 각도(같은 길의 길이)	°
T	티	**이동 시간**: 그 길을 가는 데 걸리는 시간	s
F	에프	**힘**: 물체를 가속하는 데 필요한 힘	N
m	엠	**질량**: 물체의 무게를 정하는 양	kg
a	에이	**가속도**: 1 초마다 빨라지는 정도	m/s²

개념 자세히

쌓기는 지금까지 배운 것을 모두 합친 과제다. 블록 위치에서 2 기구학 & **역기구학**으로 집는 자세와 놓는 자세를 구하고, 6-1·6-2 로 충돌 없는 길을 만들고(곧장 가면 책상에 닿을 때는 RRT 로 돌아간다), 3 미분기구학 · **궤적** 생성의 시간 곡선을 입히고, 4 로봇 동역학으로 **토크**가 모터 한계 안인지 확인한 뒤, 물리 엔진이 결과를 보여 준다. 집고 놓는 순서는 6-3 의 네 단계 그대로이고, 블록이 25 mm 이므로 한 층 오를 때마다 놓는 높이가 25 mm 씩 올라간다. 실습에서는 파랑·빨강·노랑 블록을 차례로 한 자리에 쌓는다.

같은 순서를 세 빠르기로 해 본다. 느리게는 부드러운 5차 곡선으로 최대 90°/s, 빠르게는 같은 곡선으로 최대 180°/s, 너무 빠르게는 사다리꼴 곡선으로 최대 400°/s 다. 이것은 시뮬레이터의 관절 속도 한계 200°/s 를 일부러 넘긴 계산으로, 실제로는 낼 수 없는 빠르기라 **토크**도 한계를 넘는다. 걸린 시간은 약 18.8 → 10.3 → 5.3 초로, 한 단계마다 거의 절반으로 준다. 하지만 같은 길 D(이동 거리)를 같은 모양의 곡선으로 시간 T(이동 시간)에 가면 가속도는 D / T^2 에 비례한다. 절반 시간에 가려면 가속도가 4 배가 되어야 하므로, 팔을 가속하는 데 드는 토크가 커진다. 실습의 토크 그래프에서 가장 힘든 관절의 최대 토크는 느리게 약 37%, 빠르게 약 63% 로 한계(100%) 아래에 머물지만, 너무 빠르게는 약 239% 로 한계를 크게 넘는다. 실물 모터라면 따라가지 못하거나 멈추는 동작이다.

탑이 어긋나는 것은 흔들림 때문이다. 이 시뮬레이터의 서보는 명령한 각도를 스프링처럼 따라가므로, 천천히 움직이면 잘 따라가지만 급히 서면 목표를 지나쳤다가 되돌아오며 흔들린다. 5 로봇 제어공학에서 본 **오버슈트**다. 이 흔들림이 놓는 순간 블록을 밀어, 맨 위 블록이 맨 아래 블록에서 수평으로 어긋난다(탑 기울기). 실습에서 탑 기울기는 약 0.5 · 1.8 · 8 mm 다. 아래가 조금만 어긋나도 그 위에 올린 블록은 그만큼 더 어긋나므로 오차는 위로 갈수록 쌓인다.

빠르기와 안정성은 맞바꾸는 관계다. 실제 공장에서도 **사이클 시간**과 성공률을 함께 보며 속도를 정한다. 시뮬레이션에서 통과한 동작만 실물로 보내는 것이 안전하고, 실물은 시뮬레이션과 차이가 있으므로 시뮬레이션에서 되는 가장 빠른 속도보다 여유를 두고 고른다. 이 차이는 8 로봇 학습에서 다시 만난다.

대학 · 수식

사이클 시간 = Σ (이동 + 내리기 + 잡기·놓기 + 들기)

같은 길 D 를 같은 모양의 곡선으로 시간 T 에 갈 때 최대 가속도 ∝ D / T^2 → T 가 절반이면 가속도 4 배

5차 곡선: 최대 속도 = 1.875 D / T, 최대 가속도 ≈ 5.77 D / T^2

관절 토크 τ = M(q) q̈ + (속도 항) + g(q) (가속도 항 M(q) q̈ 가 빠르기와 함께 커진다)

블록을 옆으로 가속하는 힘 F = m a, n 층 블록을 놓는 바닥 높이 = (n − 1) × 25 mm

탐구 활동 시뮬레이터 [교육] 탭 · 6-4 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 느리게

처음 조건 · 빠르기: 느리게 (최대 90°/s)

목표 느린 속도(5차 곡선, 최대 90°/s)로 세 블록을 쌓아 기준이 되는 시간 · **토크** · 기울기를 잰다.

예상 가장 느리게 쌓으면 탑은 얼마나 반듯할까? 걸린 시간과 최대 **토크**는 얼마쯤일까?

☐ 1. 반듯하고 토크는 한계 아래 ☐ 2. 기울고 토크는 한계를 넘음 ☐ 3. 반듯하지만 토크는 한계를 넘음

할 일 `느리게` 를 누른다 (5차 곡선, 최대 90°/s)

볼 것 탑이 반듯하게 서고(기울기 약 0.5 mm) 최대 **토크**가 한계의 약 37% 로 한참 아래다. 시간이 가장 오래 걸린다(약 18.8 초)

왜 느리게 움직이면 가속도가 작아 가속에 드는 **토크**가 작으므로, 가장 힘든 관절의 최대 토크도 한계의 약 37% 로 넉넉하다. 서보도 명령을 여유 있게 따라가 멈출 때 거의 흔들리지 않으므로, 블록이 제자리에 놓여 탑 기울기가 약 0.5 mm 에 그친다. 대신 18.8 초로 가장 오래 걸린다.

핵심 **느리게 → 탑 반듯, 토크 37%, 18.8 초**

2 활동 2 · 빠르게

바꾸는 것 · 빠르기: 느리게 → 빠르게 (최대 180°/s)

목표 최대 속도를 두 배로 올리면 시간 · **토크** · 기울기가 어떻게 바뀌는지 알아본다.

예상 최대 속도를 90 → 180°/s 로 두 배 올리면 걸린 시간은 절반이 될까? 최대 **토크**는 몇 배쯤 커질까?

☐ 1. 시간 거의 절반, 토크 4 배 ☐ 2. 시간 그대로, 토크 2 배 ☐ 3. 시간 거의 절반, 토크 2 배 안쪽

할 일 `빠르게` 를 누른다 (5차 곡선, 최대 180°/s)

볼 것 걸린 시간이 약 10.3 초로 거의 절반이 되고 최대 **토크**가 약 63% 로 커지지만 한계 안이다. 탑은 거의 반듯하다(약 1.8 mm)

왜 같은 5차 곡선으로 이동 시간이 절반이 되면 가속도는 $\frac{\text{이동 거리}}{\text{이동 시간}^2}$ $\frac{D}{T^2}$ 에 따라 4 배가 된다. 그런데 최대 **토크**가 4 배가 아니라 37% → 63% 로만 커진 것은, 토크에 가속 몫 말고도 빠르기와 상관없는 중력 몫이 들어 있기 때문이다. 펌프를 켜고 끄며 기다리는 시간처럼 빨라지지 않는 시간도 있어 걸린 시간은 정확히 절반이 아닌 10.3 초가 되고, 흔들림이 조금 커져 기울기는 약 1.8 mm 다.

핵심 **빠르게 → 10.3 초, 토크 63% 로 한계 안**

3 활동 3 · 너무 빠르게

바꾸는 것 · 빠르기: 빠르게 → 너무 빠르게 (사다리꼴, 최대 400°/s)

목표 모터 한계를 넘는 빠르기에서 무엇이 무너지는지 확인한다.

예상 사다리꼴 곡선으로 최대 400°/s 까지 올리면 시간은 또 절반쯤 줄까? **토크**와 탑은 어떻게 될까?

☐ 1. 시간이 줄고 탑도 반듯하다 ☐ 2. 시간은 줄지만 토크가 한계를 넘고 탑이 어긋난다 ☐ 3. 시간이 오히려 늘어난다

할 일 `너무 빠르게` 를 누른다 (사다리꼴, 최대 400°/s)

볼 것 시간은 약 5.3 초로 또 절반쯤 줄지만 최대 **토크**가 약 239% 로 한계를 넘고, 급히 설 때 팔이 흔들려 맨 위 블록이 약 8 mm 어긋난다

왜 사다리꼴 곡선은 가속을 갑자기 시작하고 끝내며 4000°/s² 의 큰 가속도를 쓰므로, 가속 **토크**가 한계의 약 239% 까지 치솟는다. 실물 모터라면 그만한 토크를 내지 못해 명령을 따라가지 못하거나 멈춘다. 또 급히 설 때 서보가 스프링처럼 목표를 지나쳤다 돌아오며 흔들리고, 놓는 순간 그 흔들림이 블록을 밀어 맨 위 블록이 약 8 mm 어긋난다.

핵심 **너무 빠르게 → 토크 239%, 탑 8 mm 어긋남**

✎ 기록표: 화면의 결과를 적어요

활동	걸린 시간(s)	최대 토크(한계 대비 %)	탑 기울기(mm)
활동 1 느리게			
활동 2 빠르게			
활동 3 너무 빠르게			

정리: 세 활동 비교하기

세 빠르기의 토크 그래프를 겹친다. 끝난 탑 셋을 나란히 놓는다. 시간·토크·기울기를 표로 본다.

알게 된 것:

정리 질문

실물 로봇에는 왜 시뮬레이션에서 되는 가장 빠른 속도보다 느리게 보낼까?

내 답:

기억할 것

빠르기와 안정성은 맞바꾸는 관계다. 시뮬레이션으로 되는 가장 빠른 속도를 찾는다. 실물에는 여유를 둔다.

현장에서는

공장에서는 사이클 시간을 줄이는 것이 곧 생산량이다. 하지만 너무 빠르면 불량과 정지가 생긴다. 모터 과부하 · 진동 · 물건 떨어뜨림 때문이다. 상자를 팔레트에 쌓는 로봇을 팔레타이징 로봇이라 한다. 이 로봇은 짐 무게에 맞춰 가속도를 낮추는 설정을 쓴다. 시뮬레이션과 시험 운전으로 속도를 확인한다. 그리고 그 속도에 여유를 두고 운영한다.

교사용 답

예상 정답: 활동 1: 1. 반듯하고 토크는 한계 아래 · 활동 2: 3. 시간 거의 절반, 토크 2 배 안쪽 · 활동 3: 2. 시간은 줄지만 토크가 한계를 넘고 탑이 어긋난다

정리 질문: 실물은 시뮬레이션과 조금 달라서, 여유를 두어야 모터 과부하나 흔들림으로 탑이 무너지지 않는다.

개념 확인 질문: 사이클 시간이란 무엇일까?

다음 차시 · 24차시 7-1 카메라와 좌표: 카메라가 본 "사진 속 위치" 가 있다. 이것을 로봇이 갈 "책상 위 위치" 로 어떻게 바꿀까?

사이클 시간 Cycle time — 작업을 한 번 하는 데 걸리는 시간. 이동·내리기·잡기·놓기·들기를 모두 더한다

궤적 trajectory — 경로 + 시간. 언제 어디에 있는지까지 정한 움직임.

관성 inertia — 움직임을 바꾸기 싫어하는 성질. 무겁고 멀리 있을수록 크다.

역기구학 inverse kinematics — 손끝을 둘 위치·자세가 주어졌을 때 관절 각도를 찾는 것. 답이 여러 개이거나 없을 수 있다.

토크 torque — 돌리는 힘. 힘 × 회전축까지의 거리(N·m).

오버슈트 overshoot — 목표를 지나쳤다가 되돌아오는 것.